x86 Windows Exploit Dev – Quick Reference

OSED Lamination Card

Registers

Reg	Size	Role	Saved
EAX	32	Return val, accumulator	Caller
EBX	32	General, base	Callee
ECX	32	Counter, fastcall arg1, `this`	Caller
EDX	32	Data, mul/div hi, fastcall arg2	Caller
ESI	32	Source index, string ops	Callee
EDI	32	Dest index, string ops	Callee
ESP	32	Stack pointer	–
EBP	32	Frame pointer (convention)	Callee

Sub-registers: EAX = AX (lo 16) = AH:AL (hi/lo 8). Same pattern for EBX/ECX/EDX.
EIP – not directly readable/writable. Changed by `call`, `ret`, `jmp`, `jcc`.
FS:[0x00] = SEH chain head. **FS:[0x30]** = PEB pointer.

EFLAGS

Flag	Bit	Meaning
CF	0	Carry / unsigned overflow
PF	2	Parity of low byte
ZF	6	Result is zero
SF	7	Result is negative (MSB)
OF	11	Signed overflow
DF	10	String direction (0=fwd)

Operand Sizes

Qualifier	Bits	Examples
BYTE PTR	8	AL, BL, CL, DL, AH…
WORD PTR	16	AX, BX, CX, DX, SI…
DWORD PTR	32	EAX, EBX, ECX, EDX…

Core Instructions

Data Movement

Instruction	Effect
`mov dst, src`	dst = src (no mem-to-mem)
`movzx dst, src`	zero-extend smaller to larger
`movsx dst, src`	sign-extend smaller to larger
`xchg a, b`	swap a and b (atomic)
`lea dst, [expr]`	dst = address expr (no mem read)
`push val`	ESP-=4; [ESP]=val
`pop dst`	dst=[ESP]; ESP+=4
`pushad`	push EAX,ECX,EDX,EBX,ESP,EBP,ESI,EDI
`popad`	pop all (skip ESP)

PUSHAD stack layout (top=lowest addr): ESP+0x00=EDI, +0x04=ESI, +0x08=EBP, +0x0C=origESP, +0x10=EBX, +0x14=EDX, +0x18=ECX, +0x1C=EAX

Arithmetic

Instruction	Effect	Flags
`add dst, src`	dst += src	OF SF ZF CF
`sub dst, src`	dst -= src	OF SF ZF CF
`inc dst`	dst += 1	OF SF ZF (not CF!)
`dec dst`	dst -= 1	OF SF ZF (not CF!)
`neg dst`	dst = 0 - dst	CF=1 unless 0
`mul src`	EDX:EAX = EAX * src (unsigned)	
`imul`	signed multiply (1/2/3 operand)	
`div src`	EAX=EDX:EAX/src, EDX=rem	
`idiv src`	signed div (cdq first!)	
`adc dst, src`	dst += src + CF	
`sbb dst, src`	dst -= src - CF	
`cdq`	EDX = sign-extend(EAX)	

Null-free zero: `xor eax,eax` then `cdq` zeros EDX (1 byte).

Logic and Bit Manipulation

Instruction	Effect
`and dst, src`	dst &= src; clears CF,OF
`or dst, src`	dst
`xor dst, src`	dst ^= src; clears CF,OF
`not dst`	dst = ~dst; no flag change
`test a, b`	a & b, set flags, discard
`shl dst, n`	left shift (= multiply by 2^n)
`shr dst, n`	logical right shift (unsigned)
`sar dst, n`	arithmetic right shift (signed)
`rol/ror dst, n`	rotate left/right
`bswap dst`	reverse byte order

Zero check: `test eax, eax` preferred over `cmp eax, 0` (shorter encoding).

Comparison and Control Flow

Instruction	Effect
`cmp a, b`	compute a-b, set flags, discard
`call target`	push EIP_next; jmp target
`ret`	EIP=[ESP]; ESP+=4
`ret N`	pop EIP, then ESP+=N
`jmp target`	unconditional jump
`loop label`	ECX–; jump if ECX!=0
`int 3`	breakpoint (0xCC)
`nop`	no-op (0x90)
`leave`	ESP=EBP; pop EBP

String Operations

All use ESI (src), EDI (dst), ECX (count for rep), DF (direction).

Instruction	Effect
`rep movsb/d`	memcpy ECX bytes/dwords [ESI]->[EDI]
`rep stosb/d`	memset ECX bytes/dwords at [EDI] with AL/EAX
`lodsb/d`	AL/EAX=[ESI]; ESI+=1/4
`repne scasb`	scan [EDI] for AL (strlen)
`repe cmpsb`	compare [ESI] vs [EDI] (strcmp)
`cld` / `std`	DF=0 (forward) / DF=1 (backward)

Windows ABI requires DF=0. Always `cld` before string ops in shellcode.

Conditional Jumps

Unsigned	Signed	Condition	Flags
JA	JG	>	CF=0, ZF=0 / ZF=0, SF=OF
JAE	JGE	>=	CF=0 / SF=OF
JB	JL	<	CF=1 / SF!=OF
JBE	JLE	<=	CF=1 or ZF=1 / ZF=1 or SF!=OF
JE	JE	==	ZF=1
JNE	JNE	!=	ZF=0
JZ	–	zero	ZF=1
JS	–	negative	SF=1
JO	–	overflow	OF=1
JC	–	carry	CF=1

Calling Conventions

Conv	Args	Cleanup	Reg args
cdecl	R-to-L stack	Caller (`add esp,N`)	None
stdcall	R-to-L stack	Callee (`ret N`)	None
fastcall	R-to-L stack	Callee (`ret N`)	ECX, EDX
thiscall	R-to-L stack	Callee (`ret N`)	ECX (this)

Stack Effects

Instruction	ESP change
`push reg`	-4
`pop reg`	+4
`call target`	-4
`ret`	+4
`ret N`	+4+N
`pushad`	-32
`popad`	+32
`sub esp, N`	-N
`add esp, N`	+N
`leave`	ESP=EBP+4

Implicit Register Usage

Instruction	Implicit regs
`mul reg`	EDX:EAX = EAX * reg
`div reg`	EAX = EDX:EAX / reg, EDX = rem
`cdq`	EDX = sign-extend(EAX)
`loop`	ECX–
`rep movsb`	[ESI]->[EDI], ECX times
`rep stosb`	AL->[EDI], ECX times
`lodsb`	AL=[ESI]; ESI++
`scasb`	cmp AL,[EDI]; EDI++
`xlat`	AL=[EBX+AL]

PEB Walking Offsets

Step	Offset
TEB -> PEB	FS:[0x30]
PEB -> Ldr	PEB + 0x0C

Step	Offset
Ldr -> InLoadOrderModuleList	Ldr + 0x0C
Ldr -> InMemoryOrderModuleList	Ldr + 0x14
Ldr -> InInitOrderModuleList	Ldr + 0x1C
Entry -> DllBase	Entry + 0x18
Entry -> BaseDllName	Entry + 0x2C

PE Export Resolution

Field	Location
e_lfanew	DllBase + 0x3C
Export Dir RVA	DllBase + e_lfanew + 0x78
NumberOfNames	ExportDir + 0x18
AddressOfFunctions	ExportDir + 0x1C
AddressOfNames	ExportDir + 0x20
AddressOfNameOrdinals	ExportDir + 0x24

SEH Chain

Field	Location
SEH chain head	FS:[0x00]
Next record	[record + 0x00]
Handler	[record + 0x04]
Terminator	Next = 0xFFFFFFFF

Page Protection Constants

Constant	Value
PAGE_NOACCESS	0x01
PAGE_READONLY	0x02
PAGE_READWRITE	0x04
PAGE_EXECUTE	0x10
PAGE_EXECUTE_READ	0x20
PAGE_EXECUTE_READWRITE	0x40
MEM_COMMIT	0x1000
MEM_RESERVE	0x2000
MEM_COMMIT + MEM_RESERVE	0x3000

DEP Bypass APIs

API	Key args
VirtualAlloc	lpAddr, dwSize=1, 0x1000, **0x40**
VirtualProtect	lpAddr, dwSize, **0x40**, lpOldProtect(writable!)
WriteProcessMemory	hProc=-1, lpBase(exec), lpBuf(src), nSize, lpWritten

Null-Avoidance Tricks

Need	Trick
Small positive (nulls)	Add negative: -0x1C = 0xFFFFFFE4
Value 0x01	`pop eax=0xFFFFFFFF; neg eax`
0x1000	0x80808080 + 0x7F7F8F80
0x40	0x80808080 + 0x7F7F7FC0
Bad byte in addr	ship addr+/-1, correct with add/sub

ROP Gadget Patterns

Gadget	Effect
`pop eax; ret`	load constant (8 bytes consumed)
`pop eax; pop ecx; ret`	load 2 constants (12 bytes)
`xchg eax, esp; ret`	stack pivot
`mov [edi], eax; ret`	store primitive
`mov [esi], eax; ret`	store primitive
`add esp, 0x20; ret`	skip 0x24 bytes
`pushad; ret`	API call setup
`mov eax, [eax]; ret`	dereference (IAT resolve)
`neg eax; ret`	negate
`push esp; pop eax; ret`	capture ESP

Core rhythm: align ESI/EDI to slot -> compute value into EAX -> `mov [esi], eax`
Pivot gotcha: `mov esp,ebp; pop ebp; ret` pops one DWORD – aim EBP 4 bytes before API slot.

Egghunter (NtAccess, ~32 bytes)

```
or dx,0xfff         ; align to page-1
inc edx             ; next page
push edx / push XX  ; syscall num
pop eax / int 0x2e  ; invoke
cmp al,5 / pop edx  ; ACCESS_VIOLATION?
je short (back)     ; yes = skip page
mov eax,"w00t"      ; tag
mov edi,edx         ; search ptr
scasd / jne (back)  ; first match?
scasd / jne (back)  ; second match?
jmp edi             ; found! execute
```

Win10 fix: syscall > 0x7F has nulls. Use `mov eax, neg(syscall); neg eax`.
Tag setup: `tag + tag + shellcode` in the payload buffer.

WinDbg Commands

Command	Purpose
`r`	Show registers
`u addr / ub / uf`	Disasm forward / backward / function
`dd addr / db / dc`	DWORDs / bytes / bytes+ASCII
`da addr / du`	ASCII string / Unicode string
`dps addr / dds`	Pointers / DWORDs with symbols
`k / kb / kv`	Call stack variants
`lm`	List modules
`!address addr`	Memory region info
`!vprot addr`	Page protection
`!teb / !peb`	Thread/Process env block
`dt type addr`	Display structure
`.exr -1 / .ecxr`	Last exception / exception context
`!analyze -v`	Verbose crash analysis
`bp addr`	Software breakpoint
`ba r4 addr`	HW read breakpoint
`ba w4 addr`	HW write breakpoint
`sxe av`	Break on access violation
`? expr`	Evaluate expression
`!nmod`	Module mitigations (narly)
`ed esp 90909090`	Write to memory
`pt / p / g`	Step-to-ret / step / go

Python Packing (struct)

Format	Type	Size	Endian
`<I`	uint32	4	Little
`<i`	int32	4	Little
`<H`	uint16	2	Little
`<h`	int16	2	Little
`<B`	uint8	1	–
`>I`	uint32	4	Big

`struct.pack("<I", 0x41414141) = b'\x41\x41\x41\x41'`